

1. Patrón OBSERVER (Para el Sistema de Alertas)

- **¿Qué hace?** Notifica automáticamente los cambios de estado de un objeto a múltiples interesados.
- **Idea para el proyecto: Notificación de Subastas o Nuevas Obras.** * Cuando un artista publica una nueva obra o inicia una subasta en el Marketplace, el Observer se encarga de avisar automáticamente a todos los compradores/seguidores interesados en ese artista sin acoplar el código.

```
# 14. PATRÓN OBSERVER (Notificar cambios de la obra a seguidores)
@app.post("/simular_notificacion_observer/{obra_id}", tags=["Comportamiento"])
def simular_alerta_obra_patron_observer(obra_id: int, evento: str = "cambio_precio"):
    obra_obj = next((o for o in sistema.lista_obras if o.id == obra_id), None)
    if not obra_obj:
        raise HTTPException(status_code=404, detail="Obra no encontrada")

    # Creamos el canal del sujeto (Observer)
    canal = GestorNotificacionesObra()

    # Suscribimos dinámicamente a los interesados
    cliente_vip = ColeccionistaSeguidor("Carlos_Bucaramanga")
    admin_sistema = AdministradorPlataforma()

    canal.suscribir(cliente_vip)
    canal.suscribir(admin_sistema)

    # Redactamos el mensaje según lo que pasó en el Marketplace
    if evento == "cambio_precio":
        mensaje = f"La obra '{obra_obj.obra}' cambió su precio comercial a ${obra_obj.precio} USD"
    elif evento == "vendida":
        mensaje = f"¡URGENTE! La pieza única '{obra_obj.obra}' ha sido vendida. Retirar de vitrinas."
```

```
# PATRÓN OBSERVER (Alertas)
# =====

class InteresadoObserver(ABC):
    @abstractmethod
    def actualizar(self, mensaje: str) -> str: pass

class ColeccionistaSeguidor(InteresadoObserver):
    def __init__(self, nombre_usuario: str):
        self.nombre = nombre_usuario

    def actualizar(self, mensaje: str) -> str:
        return f"NOTIFICACIÓN PARA CLIENTE [{self.nombre}]: {mensaje}"

class AdministradorPlataforma(InteresadoObserver):
    def actualizar(self, mensaje: str) -> str:
        return f"ALERTA DEL SISTEMA [ADMIN]: {mensaje}"

# El sujeto que será observado (Canal de Notificaciones de la Obra)
class GestorNotificacionesObra:
```

POST /simular_notificacion_observer/{obra_id} Simular Alerta Obra

Parameters

Name	Description
obra_id * required	
integer (path)	5
evento	
string (query)	vendida

Execute

200

Response body

```
{
  "obra_monitoreada": "kk1",
  "evento_ocurrido": "vendida",
  "cantidad_notificados": 2,
  "registro_de_alertas": [
    "NOTIFICACIÓN PARA CLIENTE [Carlos_Bucaramanga]: ¡URGENTE! La pieza única 'kk1' ha sido vendida. Retirar de vitrinas.",
    "ALERTA DEL SISTEMA [ADMIN]: ¡URGENTE! La pieza única 'kk1' ha sido vendida. Retirar de vitrinas."
  ]
}
```

Response headers

2. Patrón STRATEGY (Para el Cálculo Dinámico)

- **¿Qué hace?** Permite intercambiar algoritmos o lógicas de cálculo en tiempo de ejecución.
- **Idea para el proyecto: Estrategias de Descuento o Pasarelas de Pago.**
 - **Descuentos:** Podemos tener diferentes estrategias de precios según la temporada (ej: DescuentoDiaMadre del 15%, DescuentoArtistasNuevos del 10%, o SinDescuento). El sistema aplica la estrategia elegida dinámicamente al cobrar.
 - **Comisiones:** Diferentes lógicas de cobro de comisión según si el vendedor es un artista "Premium" o "Estándar".

```
# 11. PATRÓN STRATEGY (Calcular totales de pago dinámicamente)
@app.get("/calcular_pago_strategy/{obra_id}", tags=["Comportamiento"])
def calcular_pago_obra_patron_strategy(obra_id: int, metodo_pago: str = "pasarela"):
    obra_obj = next((o for o in sistema.lista_obras if o.id == obra_id), None)
    if not obra_obj:
        raise HTTPException(status_code=404, detail="Obra no encontrada")

    # Selección dinámica de la estrategia
    if metodo_pago.lower() == "pasarela":
        estrategia = PagoPasarelaEstandar()
    elif metodo_pago.lower() == "transferencia":
        estrategia = PagoTransferenciaDirecta()
    elif metodo_pago.lower() == "cripto":
        estrategia = PagoCriptoNFT()
    else:
        raise HTTPException(status_code=400, detail="Método de pago no soportado. Usa 'pasarela', 'transferencia' o 'cripto'")

    # Extraemos el precio convirtiéndolo a flotante de forma segura
    precio_base = float(obra_obj.precio)

    # Ejecutamos el algoritmo de la estrategia elegida
```

```
class EstrategiaPago(ABC):
    @abstractmethod
    def calcular_total(self, precio_base: float) -> dict: pass

class PagoPasarelaEstandar(EstrategiaPago):
    def calcular_total(self, precio_base: float) -> dict:
        # Regla: 10% de comisión para la plataforma
        comision = round(precio_base * 0.10, 2)
        total = round(precio_base + comision, 2)
        return {
            "metodo": "Pasarela de Pago Estándar (Tarjeta/PSE)",
            "precio_obra": precio_base,
            "comision_plataforma": comision,
            "cargos_extra": 0.0,
            "total_a_pagar": total
        }

class PagoTransferenciaDirecta(EstrategiaPago):
    def calcular_total(self, precio_base: float) -> dict:
        # Regla: 2% de comisión + $5 USD fijos por manejo seguro de la plata
```

GET

/calcular_pago_strategy/{obra_id}

Calcular Pago Obra Patron Strategy

Parameters

Name	Description
obra_id required	
integer (path)	5
metodo_pago	
string (query)	pasarela

Execute

Responses

Code

Details

200

Response body

```
{
  "obra_id": 5,
  "titulo_obra": "kk1",
  "artista": "pero",
  "liquidacion_detallada": {
    "metodo": "Pasarela de Pago Estándar (Tarjeta/PSE)",
    "precio_obra": 1000,
    "comision_plataforma": 100,
    "cargos_extra": 0,
    "total_a_pagar": 1100
  }
}
```

Response headers

```
content-length: 219
content-type: application/json
date: Wed, 20 May 2026 14:27:11 GMT
server: uvicorn
```

Responses

3. Patrón COMMAND (Para las Acciones de Compra y Carrito)

- **¿Qué hace?** Encapsula una solicitud como un objeto, lo que permite parametrizar operaciones, hacer colas de peticiones o implementar la función de "deshacer" (Undo).
- **Idea para el proyecto: Orden de Compra / Transacciones.**
 - Cada acción de compra (ej: ComprarObraCommand) se convierte en un objeto independiente. Si la transacción falla a mitad de camino por problemas de red, el comando permite hacer un *rollback* (deshacer) para liberar el stock de la obra única inmediatamente.

```
# --- NUEVO: PATRÓN COMMAND (Transacciones) -
# =====

class ComandoTransaccion(ABC):
    @abstractmethod
    def ejecutar(self) -> str: pass

    @abstractmethod
    def deshacer(self) -> str: pass

class ComandoComprarObra(ComandoTransaccion):
    def __init__(self, inventario, obra_id: int):
        self.inventario = inventario
        self.obra_id = obra_id
        self.obra_afectada = None

    def ejecutar(self) -> str:
        # Buscamos la obra en el Singleton
        obra_obj = next((o for o in self.inventario.lista_obras if o.id == self.obra_id), None)
        if not obra_obj:
            return "Error: Obra no encontrada."

# 15. PATRÓN COMMAND (Ejecutar compra encapsulada)
@app.post("/procesar_compra_command/{obra_id}", tags=["Comportamiento"])
def comprar_con_comando(obra_id: int):
    global ultimo_comando_ejecutado

    # Encapsulamos la petición en un objeto comando
    comando = ComandoComprarObra(sistema, obra_id)
    resultado = comando.ejecutar()

    if "Error" in resultado:
        raise HTTPException(status_code=400, detail=resultado)

    # Guardamos el objeto comando en memoria para poder revertirlo si se requiere
    ultimo_comando_ejecutado[obra_id] = comando
    sistema.guardar_datos()

    return {
        "transaccion": "Comando Recibido",
        "resultado_ejecucion": resultado,
        "ayuda": "Si simulas un fallo en el pago, puedes llamar al endpoint de deshacer."
    }
```

```
# 16. PATRÓN COMMAND (Deshacer / Rollback de la compra)
@app.post("/deshacer_compra_command/{obra_id}", tags=["Comportamiento"])
def deshacer_con_comando(obra_id: int):
    global ultimo_comando_ejecutado

    comando = ultimo_comando_ejecutado.get(obra_id)
    if not comando:
        raise HTTPException(status_code=400, detail="No hay una transacción")

    # Llamamos al método deshacer del objeto guardado
    resultado_rollback = comando.deshacer()

    # Limpiamos el historial de esa obra
    del ultimo_comando_ejecutado[obra_id]
    sistema.guardar_datos()

    return {
        "transaccion": "Rollback Exitoso",
        "detalle_sistema": resultado_rollback
    }
```

000

Details

200

Response body

```
{
  "transaccion": "Comando Recibido",
  "resultado_ejecucion": "Éxito: La obra ha sido adquirida y pasó a estado VENDIDA.",
  "ayuda": "Si simulas un fallo en el pago, puedes llamar al endpoint de deshacer."
}
```

Response headers

```
content-length: 199
content-type: application/json
date: Wed,20 May 2026 15:33:50 GMT
server: uvicorn
```

responses

200

Details

200

Response body

```
{
  "transaccion": "Rollback Exitoso",
  "detalle_sistema": "Rollback Transaccional: Se restauró la obra 'kkl' a Disponible."
}
```

Response headers

```
content-length: 119
content-type: application/json
date: Wed,20 May 2026 15:34:30 GMT
server: uvicorn
```

Responses

POST

/procesar_compra_command/{obra_id}

Comprar Con Comando

Parameters

Name	Description
obra_id * required	
integer	5
(path)	

Execute

Responses

4. Patrón STATE (Para el Ciclo de Vida de la Obra)

- **¿Qué hace?** Permite que un objeto altere su comportamiento cuando su estado interno cambia.
- **Idea para el proyecto: Flujo de la Obra en el Marketplace.**
 - Una obra de arte no se vende de la nada; pasa por estados: Disponible $\rightarrow$ EnSubasta $\rightarrow$ Reservada (en carrito) $\rightarrow$ Vendida.
 - Si la obra está en estado Vendida, el método ofertar() o comprar() lanzará un error automáticamente. Si está EnSubasta, el comportamiento cambia para recibir pujas en vez de compra directa.

```
# 10. PATRÓN STATE (Simular la compra y cambio de comportamiento)
@app.post("/gestionar_ciclo_vida_state/{obra_id}", tags=["Comportamiento"])
def comprar_o_descontar_obra_patron_state(obra_id: int, accion: str = "comprar", descuento_porcentaje: float = 10.0):
    obra_obj = next((o for o in sistema.lista_obras if o.id == obra_id), None)
    if not obra_obj:
        raise HTTPException(status_code=404, detail="Obra no encontrada en el catálogo")

    # Comprobamos qué acción quiere ejecutar el cliente en el Marketplace
    if accion.lower() == "comprar":
        resultado = obra_obj.comprar_obra()
    elif accion.lower() == "descuento":
        resultado = obra_obj.rebajar_precio(descuento_porcentaje)
    else:
        raise HTTPException(status_code=400, detail="Acción no válida. Usa 'comprar' o 'descuento'")

    sistema.guardar_datos() # Persistimos el nuevo estado con el Singleton

    return {
        "obra": obra_obj.obra,
        "artista": obra_obj.artista,
        "precio_actual": obra_obj.precio,
        "stock_restante": obra_obj.stock
```

```
class EstadoObra(ABC):
    @abstractmethod
    def cambiar_disponibilidad(self, obra_contexto) -> str: pass

    @abstractmethod
    def aplicar_oferta(self, obra_contexto, porcentaje_descuento: float) -> str: pass

class EstadoDisponible(EstadoObra):
    (parameter) obra_contexto: Any

    def cambiar_disponibilidad(self, obra_contexto) -> str:
        obra_contexto.estado_actual = EstadoVendido()
        obra_contexto.stock = 0 # Regla de negocio: si se vende, stock a 0
        return "Éxito: La obra ha sido adquirida y pasó a estado VENDIDA."

    def aplicar_oferta(self, obra_contexto, porcentaje_descuento: float) -> str:
        # Permite aplicar descuentos si está disponible
        precio_original = obra_contexto.precio
        obra_contexto.precio = round(precio_original * (1 - (porcentaje_descuento / 100)), 2)
        return f"Descuento aplicado con éxito. Precio anterior: ${precio_original} USD, Nuevo precio: ${obra_contexto.precio} USD"
```

Server response

Code

Details

200

Response body

```
{
  "galeria": [
    {
      "id": 2,
      "obra": "sdasad",
      "artista": "fsdfs",
      "stock": 2,
      "precio": 3000,
      "categoria": "fisico",
      "certificado": true,
      "empaque_regalo": true,
      "edicion": 1,
      "seguro": "asegurado por 234 usd",
      "sello": "ORIGINAL-D687AB13",
      "estado_actual": {},
      "dimensiones": "3cm"
    },
    {
      "id": 3
    }
  ]
}
```

POST /gestionar_ciclo_vida_state/{obra_id} Comprar O Descontar Obra Patron State

Parameters

Name	Description
obra_id * required	
integer (path)	2
accion	
string (query)	descuento
descuento_porcentaje	
number (query)	10

Execute

Code

Details

200

Response body

```
{
  "obra": "sdasad",
  "artista": "fsdfs",
  "precio_actual": 2700,
  "stock_restante": 2,
  "estado_en_sistema": "EstadoDisponible",
  "resultado_operacion": "Descuento aplicado con éxito. Precio anterior: $3000 USD, Nuevo precio: $2700.0 USD."
}
```

Response headers

Code

Details

200

Response body

```
{
  "obra": "sdasad",
  "artista": "fsdfs",
  "precio_actual": 2700,
  "stock_restante": 0,
  "estado_en_sistema": "EstadoVendido",
  "resultado_operacion": "Éxito: La obra ha sido adquirida y pasó a estado VENDIDA."
}
```

Response headers

1. Patrón STATE (Estado)

- **Qué hace en teoría:** Permite que un objeto altere su comportamiento cuando cambia su estado interno, pareciendo que cambia de clase.
- **Qué hace en nuestro proyecto:** Controla el **ciclo de vida comercial de las obras de arte** (Físicas o Digitales). Una obra nace en EstadoDisponible. Si se encuentra en este estado, el sistema permite aplicarle descuentos y realizar la compra. Al ejecutarse la venta, el objeto muta internamente a EstadoVendido. A partir de ese momento, el mismo objeto bloquea automáticamente cualquier intento de compra posterior o rebaja de precio, protegiendo las reglas de negocio del Marketplace sin llenar el código de condicionales if/else repetitivos.

2. Patrón STRATEGY (Estrategia)

Qué hace en teoría: Define un conjunto de algoritmos mutuamente intercambiables, encapsula cada uno de ellos y los hace independientes del cliente que los utiliza.

Qué hace en nuestro proyecto: Gestiona la liquidación dinámica de pagos y comisiones en tiempo de ejecución. Dependiendo del método de pago que elija el comprador en el carrito de compras (pasarela, transferencia o cripto), el sistema intercambia el algoritmo de cálculo instantáneamente.

La estrategia de Pasarela aplica un 10% de comisión fija.

La de Transferencia cobra un 2% más un cobro logístico de \$5 USD.

La de Cripto/NFT descarta comisiones porcentuales y aplica una tarifa plana de red (Gas Fee) de \$15 USD.

Esto permite expandir nuevos métodos de pago en el futuro sin modificar las clases base.

3. Patrón OBSERVER (Observador)

Qué hace en teoría: Define una dependencia de uno a muchos entre objetos, de forma que cuando un objeto cambia de estado, todos sus dependientes son notificados automáticamente.

Qué hace en nuestro proyecto: Centraliza el sistema reactivo de alertas y notificaciones del Marketplace. Funciona como un canal de suscripción. Cuando una pieza de arte sufre una actualización crítica (como una reducción de precio o el cambio a estado vendido), el gestor de la obra (Subject) despacha una notificación automática en bloque a todos los interesados suscritos (Observers), tales como el ColeccionistaSeguidor que desea cazar ofertas y el AdministradorPlataforma que monitorea las transacciones, garantizando un desacoplamiento total.

4. Patrón COMMAND (Comando)

Qué hace en teoría: Encapsula una petición como un objeto, permitiendo parametrizar a los clientes con diferentes peticiones, encolar solicitudes y soportar operaciones que se pueden deshacer.

Qué hace en nuestro proyecto: Asegura la integridad transaccional y el control de stock. Cada solicitud de compra se transforma en un objeto independiente (ComandoComprarObra). Este comando encapsula la lógica para validar el stock disponible en el inventario global (Singleton) y ejecutar la compra. Su mayor ventaja es que implementa un método deshacer(). Si la pasarela de pago externa reporta una caída o un saldo insuficiente a mitad del proceso, el sistema invoca este método para realizar un rollback automático, devolviendo el stock a la vitrina y restaurando el estado a disponible inmediatamente.